In [1]:
```python
from pdf2image import convert_from_path
from IPython.display import display

images = convert_from_path(r"C:\Users\miles\BYU-Idaho\11th Semester\336 Advanced
for img in images:
    display(img)
```

Physics 336
HW8

1. Consider 4 measured data points:

$$x = \{0.500,\ 1.000,\ 1.500,\ 2.000\} \qquad \text{(with negligible uncertainty)}$$
$$y = \{6.44,\ 8.97,\ 9.44,\ 8.12\} \pm 0.06$$

   (a) Construct a matrix representation for a quadratic fit,

$$y = a_0 + a_1 x + a_2 x^2$$

      i. Use this matrix representation to find the least squares fit.
      ii. Calculate the error matrix, and find the uncertainties in the least-squares-fit parameters.
      iii. Calculate χ^2, and assess if the given uncertainty in y is reasonable.
      iv. Calculate the fitted value for $x = 2.50 \pm 0.02$, and its uncertainty using just the uncertainties in a_0 and a_1.
      v. Calculate the uncertainty in the fitted value for $x = 2.50 \pm 0.02$ using the full covariance matrix.

   (b) Construct a matrix representation for a trigonometric fit,

$$y = a_0 + a_1 \sin(\pi x/4) + a_2 \sin(2\pi x/4)$$

      i. Use this matrix representation to find the least squares fit.
      ii. Calculate the error matrix, and find the uncertainties in the least-squares-fit parameters.
      iii. Calculate χ^2, and assess if the given uncertainty in y is reasonable.
      iv. Calculate the fitted value for $x = 2.50 \pm 0.02$, and its uncertainty using just the uncertainties in a_0 and a_1.
      v. Calculate the uncertainty in the fitted value for $x = 2.50 \pm 0.02$ using the full covariance matrix.

For Problems 2 - 5: Do at least 1 problem using Mathematica, and do at least 1 problem using Python.

2. A cart slides down an inclined air track that is not quite frictionless.

   (a) Show that the acceleration is expected to be $a = g(\sin\theta - \mu\cos\theta)$, with *down* the incline defined to be positive. Use this result to find the distance x the cart slides as a function of time t.

   (b) The angle of inclination is varied by placing blocks of differing thickness, h, under one foot of the track (the other foot is 1.000 m along the track). Take the dependent variable to be $z = 2x/t^2$ (explain why this is appropriate), and determine an expected form for z as a function of h.

(c) The time that it takes for the cart to slide a distance $x = 1.500$ m is recorded.

$$h(cm) = \{2.00, 4.00, 6.00, 8.00, 10.00\}$$
$$t(s) = \{7.9, 3.5, 2.6, 2.2, 1.9\} \pm 0.2$$

Find a best fit to this data, and use it to calculate the coefficient of friction μ with its uncertainty, and the acceleration due to gravity g with its uncertainty.

3. A symmetrical object is dropped from rest a height y above the floor. Air drag is not negligible, and is approximated as quadratic in velocity, $\frac{1}{2}\rho ACv^2$. For this case, it is convenient to take down to be positive.

   (a) Construct a model for the distance fallen as a function of time.

      i. Begin by finding the acceleration as a function of velocity. Define an appropriate terminal velocity, v_{ter}.

      ii. Separate variables, and integrate to find velocity as a function of time. Recall that $\frac{d}{du}\tanh^{-1} u = \frac{1}{1-u^2}$

      iii. Integrate velocity to find position as a function of time.

   (b) The object is dropped from varying heights, beginning at 50.0 cm and increasing by 20.0-cm increments to 2.10 m. The fall time is measured, with results:

$$y(\text{m}) = \{0.500, 0.700, 0.900, 1.100, 1.300, 1.500, 1.700, 1.900, 2.100\}$$
$$t(\text{s}) = \{0.34, 0.40, 0.44, 0.49, 0.56, 0.60, 0.64, 0.69, 0.72\} \pm 0.02 \text{ s each}$$

   Make an initial guess for v_{ter}, and make a plot of measured values compared to model values. Calculate χ^2 for your guess. Now update your guess several times, to find an approximate minimum for χ^2.

   (c) Fit the data above to your model from part (a)(iii), and find v_{ter}. Compare to your result in (b).

4. A result y is expected to be a Gaussian distribution of x with an unknown amplitude A, mean μ, and standard deviation σ. Measured values give

$$x = \{5.0, 5.4, 5.8, 6.2, 6.6, 7.0, 7.4, 7.8, 8.2, 8.6, 9.0, 9.4, 9.8, 10.2\}$$
$$y = \{0.22, 0.82, 2.10, 4.35, 7.56, 10.60, 12.38, 11.70, 9.18, 5.85, 3.11, 1.35, 0.44, 0.15\}$$

   Fit the data above to a Gaussian to determine A, μ, and σ.

5. The activity of a short-lived radioisotope is measured in the presence of an unknown background. Activity is measured in 30-second intervals for 20 minutes.

$$t(\text{min}) = \{0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0, 5.5, 6.0, 6.5, 7.0, 7.5, 8.0, 8.5, 9.0, 9.5, 10.0, 10.5, 11.0, 11.5, 12.0, 12.5, 13.0, 13.5, 14.0, 14.5, 15.0, 15.5, 16.0, 16.5, 17.0, 17.5, 18.0, 18.5, 19.0, 19.5, 20.0\}$$
$$\text{counts} = \{816, 733, 660, 593, 539, 498, 440, 407, 374, 334, 312, 279, 250, 237, 218, 187, 179, 168, 163, 144, 142, 127, 117, 118, 108, 98, 85, 86, 90, 81, 74, 71, 73, 57, 56, 56, 62, 58, 62, 61\}$$

Fit this data to find the half life of this isotope and the background.

1

1.a

In [41]:
```
from sympy import Matrix, symbols, Eq

x=[0.5,1.0,1.5,2.0]
y,dy=[6.44,8.97,9.44,8.12],0.06
a0,a1,a2 = symbols("a0 a1 a2")
A=Matrix([a0,a1,a2])
x1=Matrix([[1,i,i**2] for i in x])
y1=Matrix([y]).T
Eq(y1,x1*A)
```

Out[41]:

$$\begin{bmatrix} 6.44 \\ 8.97 \\ 9.44 \\ 8.12 \end{bmatrix} = \begin{bmatrix} a_0 + 0.5a_1 + 0.25a_2 \\ a_0 + 1.0a_1 + 1.0a_2 \\ a_0 + 1.5a_1 + 2.25a_2 \\ a_0 + 2.0a_1 + 4.0a_2 \end{bmatrix}$$

1.a.i

In [42]:
```
from sympy import Matrix, symbols, Eq

x=[0.5,1.0,1.5,2.0]
y,dy=[6.44,8.97,9.44,8.12],0.06
a0,a1,a2 = symbols("a0 a1 a2")
A=Matrix([a0,a1,a2])
x1=Matrix([[1,i,i**2] for i in x])
y1=Matrix([y]).T
Eq(A,((x1.T*x1).inv()*x1.T*y1))
```

Out[42]:

$$\begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 2.0525 \\ 10.727 \\ -3.85 \end{bmatrix}$$

1.a.ii

In [50]:
```
from sympy import Matrix, symbols, Eq

x=[0.5,1.0,1.5,2.0]
y,dy=[6.44,8.97,9.44,8.12],0.06
a0,a1,a2 = symbols("a0 a1 a2")
e,e11,e12,e13,e21,e22,e23,e31,e32,e33 = symbols("e e11 e12 e13, \
                                                 e21 e22 e23, \
                                                 e31 e32 e33")
sigma_a0, sigma_a1, sigma_a2 = symbols("sigma_a0 sigma_a1 sigma_a2")
e=Matrix([[e11,e12,e13],[e21,e22,e23],[e31,e32,e33]])
A=Matrix([a0,a1,a2])
sigma=Matrix([sigma_a0, sigma_a1, sigma_a2])
x1=Matrix([[1,i,i**2] for i in x])
y1=Matrix([y]).T
C=(x1.T*x1).inv()*dy**2
display(Eq(e,C))

uncert=Matrix([sqrt(C[i,i]) for i in range(3)])

display(Eq(sigma,uncert))
```

$$\begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix} = \begin{bmatrix} 0.0279 & -0.0486 & 0.018 \\ -0.0486 & 0.09288 & -0.036 \\ 0.018 & -0.036 & 0.0144 \end{bmatrix}$$

$$\begin{bmatrix} \sigma_{a0} \\ \sigma_{a1} \\ \sigma_{a2} \end{bmatrix} = \begin{bmatrix} 0.167032930884901 \\ 0.304762202380807 \\ 0.12 \end{bmatrix}$$

1.a.iii

In [78]:
```
from sympy import Matrix, symbols, Eq

x=[0.5,1.0,1.5,2.0]
y,dy=[6.44,8.97,9.44,8.12],0.06
x1=Matrix([[1,i,i**2] for i in x])
y1=Matrix([y]).T
best_fit=(x1.T*x1).inv()*x1.T*y1
y_hat=x1*best_fit
differences=y1-y_hat

chi_squared=(differences.T*differences)[0]/dy**2
print(f"Chi_squared: {chi_squared.evalf(5)}")
```

```
Chi_squared: 1.0125
```

Because Chi_squared is around 1, it is pretty good, but we could change our sigma value to get a better one. Trial and error says it should be around ~0.0604. So yeah, pretty good

1.a.iv

In [160…

```
from sympy import Matrix, symbols, Eq

x=[0.5,1.0,1.5,2.0]
y,dy=[6.44,8.97,9.44,8.12],0.06
a0,a1,a2 = symbols("a0 a1 a2")
e,e11,e12,e13,e21,e22,e23,e31,e32,e33 = symbols("e e11 e12 e13, \
                                                 e21 e22 e23, \
                                                 e31 e32 e33")
sigma_a0, sigma_a1, sigma_a2 = symbols("sigma_a0 sigma_a1 sigma_a2")
e=Matrix([[e11,e12,e13],[e21,e22,e23],[e31,e32,e33]])
A=Matrix([a0,a1,a2])
sigma=Matrix([sigma_a0, sigma_a1, sigma_a2])
x1=Matrix([[1,i,i**2] for i in x])
y1=Matrix([y]).T
C=(x1.T*x1).inv()*dy**2

xi,dxi=2.50,0.02

best_fit=(x1.T*x1).inv()*x1.T*y1
x=Matrix([1,xi,xi*xi]).T
y=(x*best_fit).evalf(3)

d_a0=C[0,0]**2
d_a1=(xi*sqrt(C[1,1]))**2
d_a2=((best_fit[1])*dxi)**2

dy=sqrt(d_a0+d_a1+d_a2)
print(f"Fitted value for x={xi}: {y[0].evalf(2)} +- {dy.evalf(1)} ")
```

```
Fitted value for x=2.5: 4.8 +- 0.8
```

1.a.v

In [154…

```
from sympy import Matrix, symbols, Eq

x=[0.5,1.0,1.5,2.0]
y,dy=[6.44,8.97,9.44,8.12],0.06
a0,a1,a2 = symbols("a0 a1 a2")
e,e11,e12,e13,e21,e22,e23,e31,e32,e33 = symbols("e e11 e12 e13, \
                                                 e21 e22 e23, \
                                                 e31 e32 e33")
sigma_a0, sigma_a1, sigma_a2 = symbols("sigma_a0 sigma_a1 sigma_a2")
e=Matrix([[e11,e12,e13],[e21,e22,e23],[e31,e32,e33]])
A=Matrix([a0,a1,a2])
sigma=Matrix([sigma_a0, sigma_a1, sigma_a2])
x1=Matrix([[1,i,i**2] for i in x])
y1=Matrix([y]).T
C=(x1.T*x1).inv()*dy**2

xi,dxi=2.50,0.02

best_fit=(x1.T*x1).inv()*x1.T*y1
x=Matrix([1,xi,xi*xi]).T
y=(x*best_fit).evalf(3)

d_a0=C[0,0]**2
d_a1=(xi*sqrt(C[1,1]))**2
d_a2=((best_fit[1]+2*best_fit[2]*xi)*dxi)**2

dy=sqrt(d_a0+d_a1+d_a2)
print(f"Fitted value for x={xi} using full covariance matrix:\
    {y[0].evalf(2)} +- {dy.evalf(1)} ")
```

```
Fitted value for x=2.5 using full covariance matrix:    4.8 +- 0.8
```

1.b

In [119…

```
from sympy import Matrix, symbols, Eq, pi, sin

x=[0.5,1.0,1.5,2.0]
y,dy=[6.44,8.97,9.44,8.12],0.06
a0,a1,a2 = symbols("a0 a1 a2")
A=Matrix([a0,a1,a2])
x1=Matrix([[1,sin(pi*i/4),sin(2*pi*i/4)] for i in x])
y1=Matrix([y]).T
Eq(y1,x1*A)
```

Out[119]:

$$\begin{bmatrix} 6.44 \\ 8.97 \\ 9.44 \\ 8.12 \end{bmatrix} = \begin{bmatrix} a_0 + a_1\sqrt{\frac{1}{2} - \frac{\sqrt{2}}{4}} + \frac{\sqrt{2}a_2}{2} \\ a_0 + \frac{\sqrt{2}a_1}{2} + a_2 \\ a_0 + a_1\sin(0.375\pi) + a_2\sin(0.75\pi) \\ a_0 + a_1 \end{bmatrix}$$

1.b.i

In [125…

```
from sympy import Matrix, symbols, Eq, sin

x=[0.5,1.0,1.5,2.0]
y,dy=[6.44,8.97,9.44,8.12],0.06

a0,a1,a2 = symbols("a0 a1 a2")
A=Matrix([a0,a1,a2])

x1=Matrix([[1,sin(pi*i/4).evalf(),sin(2*pi*i/4).evalf()] for i in x])
y1=Matrix(y)
A_fit=(x1.T*x1).inv()*x1.T*y1
Eq(A,A_fit)
```

Out[125]:

$$\begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 2.56942857523613 \\ 5.54885709876044 \\ 2.47279819330748 \end{bmatrix}$$

1.b.ii

In [128…

```
from sympy import Matrix, symbols, Eq

x=[0.5,1.0,1.5,2.0]
y,dy=[6.44,8.97,9.44,8.12],0.06
a0,a1,a2 = symbols("a0 a1 a2")
e,e11,e12,e13,e21,e22,e23,e31,e32,e33 = symbols("e e11 e12 e13, \
                                                 e21 e22 e23, \
                                                 e31 e32 e33")
sigma_a0, sigma_a1, sigma_a2 = symbols("sigma_a0 sigma_a1 sigma_a2")
e=Matrix([[e11,e12,e13],[e21,e22,e23],[e31,e32,e33]])
A=Matrix([a0,a1,a2])
sigma=Matrix([sigma_a0, sigma_a1, sigma_a2])
x1=Matrix([[1,sin(pi*i/4).evalf(),sin(2*pi*i/4).evalf()] for i in x])
y1=Matrix(y)
C=((x1.T*x1).inv()*dy**2).evalf(3)
display(Eq(e,C))

uncert=Matrix([sqrt(C[i,i]) for i in range(3)])

display(Eq(sigma,uncert))
```

$$\begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix} = \begin{bmatrix} 0.0236 & -0.0211 & -0.0113 \\ -0.0211 & 0.0219 & 0.00758 \\ -0.0113 & 0.00758 & 0.00925 \end{bmatrix}$$

$$\begin{bmatrix} \sigma_{a0} \\ \sigma_{a1} \\ \sigma_{a2} \end{bmatrix} = \begin{bmatrix} 0.154 \\ 0.148 \\ 0.0962 \end{bmatrix}$$

1.b.iii

In [139…

```
from sympy import Matrix, symbols, Eq

x=[0.5,1.0,1.5,2.0]
y,dy=[6.44,8.97,9.44,8.12],0.06
x1=Matrix([[1,sin(pi*i/4).evalf(),sin(2*pi*i/4).evalf()] for i in x])
y1=Matrix(y)
best_fit=(x1.T*x1).inv()*x1.T*y1
y_hat=x1*best_fit
differences=y1-y_hat

chi_squared=(differences.T*differences)[0]/dy**2
print(f"Chi_squared: {chi_squared.evalf(5)}")
```

```
Chi_squared: 0.011599
```

Because Chi_squared is much smaller than 1, it is OVER ESTIMATED, but we could change our sigma value to get a better one. Trial and error says it should be around ~0.0064. The trigonometric fit is a much better fit

1.b.iv

In [157…

```
from sympy import Matrix, symbols, Eq

x=[0.5,1.0,1.5,2.0]
y,dy=[6.44,8.97,9.44,8.12],0.06
a0,a1,a2 = symbols("a0 a1 a2")
e,e11,e12,e13,e21,e22,e23,e31,e32,e33 = symbols("e e11 e12 e13, \
                                                 e21 e22 e23, \
                                                 e31 e32 e33")
sigma_a0, sigma_a1, sigma_a2 = symbols("sigma_a0 sigma_a1 sigma_a2")
e=Matrix([[e11,e12,e13],[e21,e22,e23],[e31,e32,e33]])
A=Matrix([a0,a1,a2])
sigma=Matrix([sigma_a0, sigma_a1, sigma_a2])
x1=Matrix([[1,sin(pi*i/4).evalf(),sin(2*pi*i/4).evalf()] for i in x])
y1=Matrix(y)
C=(x1.T*x1).inv()*dy**2

xi,dxi=2.50,0.02

best_fit=(x1.T*x1).inv()*x1.T*y1
x=Matrix([1,xi,xi*xi]).T
y=(x*best_fit).evalf(3)

d_a0=C[0,0]**2
d_a1=(xi*sqrt(C[1,1]))**2
d_a2=((best_fit[1]+2*best_fit[2]*xi)*dxi)**2

dy=sqrt(d_a0+d_a1)
print(f"Fitted value for x={xi}: {y[0].evalf(3)} +- {dy.evalf(1)} ")
```

```
Fitted value for x=2.5: 31.9 +- 0.4
```

1.b.v

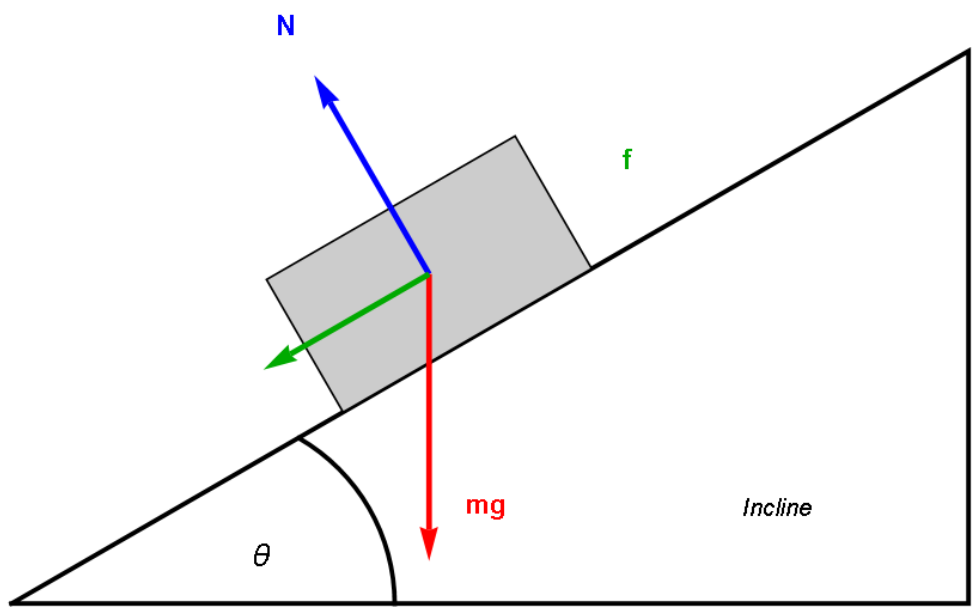
In [158…

```
from sympy import Matrix, symbols, Eq

x=[0.5,1.0,1.5,2.0]
y,dy=[6.44,8.97,9.44,8.12],0.06
a0,a1,a2 = symbols("a0 a1 a2")
e,e11,e12,e13,e21,e22,e23,e31,e32,e33 = symbols("e e11 e12 e13, \
                                                 e21 e22 e23, \
                                                 e31 e32 e33")
sigma_a0, sigma_a1, sigma_a2 = symbols("sigma_a0 sigma_a1 sigma_a2")
e=Matrix([[e11,e12,e13],[e21,e22,e23],[e31,e32,e33]])
A=Matrix([a0,a1,a2])
sigma=Matrix([sigma_a0, sigma_a1, sigma_a2])
x1=Matrix([[1,sin(pi*i/4).evalf(),sin(2*pi*i/4).evalf()] for i in x])
y1=Matrix(y)
C=(x1.T*x1).inv()*dy**2

xi,dxi=2.50,0.02

best_fit=(x1.T*x1).inv()*x1.T*y1
x=Matrix([1,xi,xi*xi]).T
y=(x*best_fit).evalf(3)

d_a0=C[0,0]**2
d_a1=(xi*sqrt(C[1,1]))**2
d_a2=((best_fit[1]+2*best_fit[2]*xi)*dxi)**2

dy=sqrt(d_a0+d_a1+d_a2)
print(f"Fitted value for x={xi} using full covariance matrix:\
    {y[0].evalf(2)} +- {dy.evalf(1)} ")
```

```
Fitted value for x=2.5 using full covariance matrix:    32 +- 0.5
```

2

2.a



$$\sum F_x = mg\sin\theta - f = ma$$

$$mg\sin\theta - \mu N = ma$$

$$\sum F_y = N - mg\cos\theta = 0$$

$$N = mg\cos\theta$$

$$mg\sin\theta - \mu(mg\cos\theta) = ma$$

$$m(g\sin\theta - \mu g\cos\theta) = ma$$

```
$$\Large{a = g(\sin \theta - \mu \cos \theta)} $$
```

$$v(t) = \int a\,dt$$

$$v(t) = \int g(\sin\theta - \mu\cos\theta)\,dt$$

$$v(t) = g(\sin\theta - \mu\cos\theta)t + v_0$$

$$x(t) = \int v(t)\,dt$$

$$x(t) = \int \left[g(\sin\theta - \mu\cos\theta)t + v_0\right]\,dt$$

$$x(t) = \frac{1}{2}g(\sin\theta - \mu\cos\theta)t^2 + v_0 t + x_0$$

$$x(t) = \frac{1}{2}g(\sin\theta - \mu\cos\theta)t^2$$

2.b

$z = 2x/t^2$ is important because it is just the equation for a.

$$x(t) = x_0 + v_0 x + \frac{1}{2}at^2$$

Where $x_0 = 0$ and $v_0 = 0$, leaving $x = \frac{1}{2}at^2$

Rearranged, $a = 2x/t^2 = z$

$$a = g(sin\theta - \mu cos\theta) = \frac{2x}{t^2}$$

$$sin\theta = \frac{h}{x} = h$$

$$cos\theta = \sqrt{1 - h^2}$$

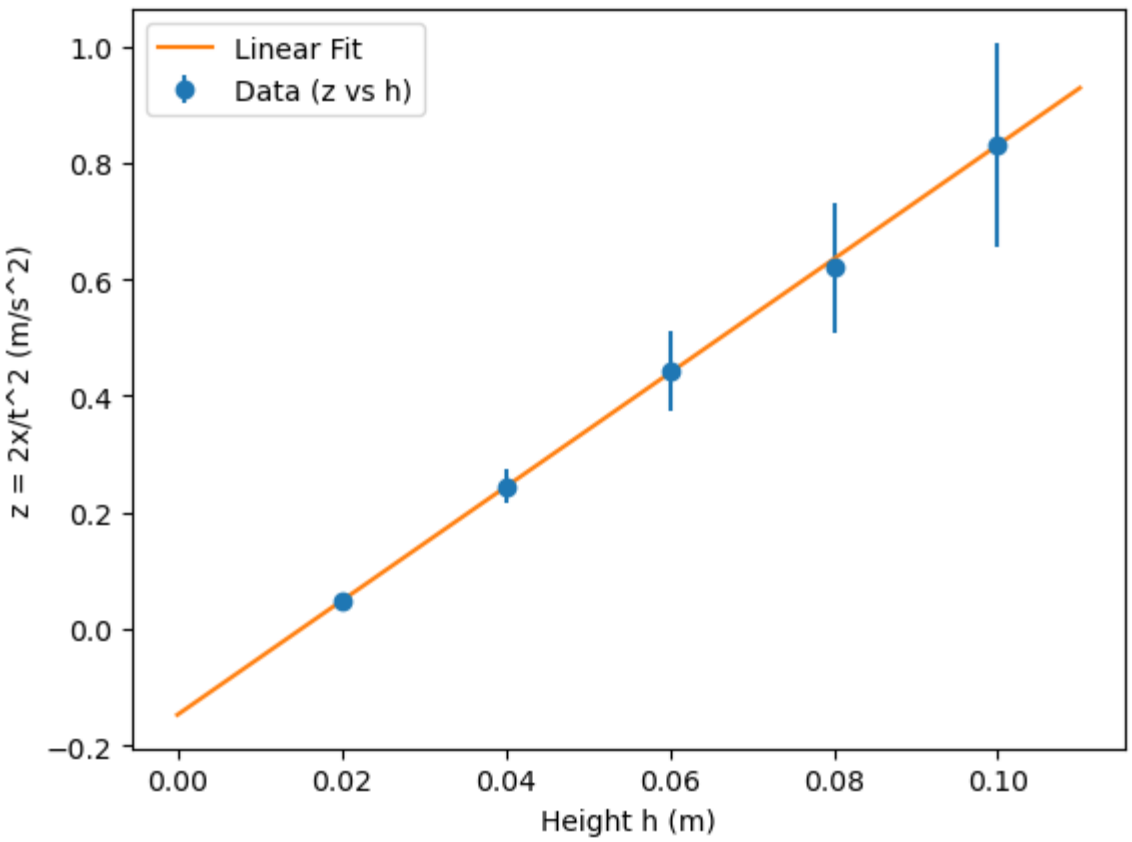
$$z = g(h - \mu\sqrt{1 - h^2})$$

2.c

In [200…

```
import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt

h = np.array([2.00, 4.00, 6.00, 8.00, 10.00]) / 100  # Convert cm to meters
t = np.array([7.9, 3.5, 2.6, 2.2, 1.9])
dt = 0.2
x_dist = 1.500

z = (2 * x_dist) / (t**2)
dz = (4 * x_dist / t**3) * dt

def physical_model(h, g, mu):
    return g*(h-mu*np.sqrt(1-h**2))

# Fit the data
guess=[9.8, 0.05]
popt, pcov = curve_fit(physical_model, h, z, p0=guess, sigma=dz)

g_fit, mu_fit = popt
g_err, mu_err = np.sqrt(np.diag(pcov))

print(f"Gravity g: {g_fit:.2f} ± {g_err:.2f} m/s^2")
print(f"Friction mu: {mu_fit:.5f} ± {mu_err:.5f}")

# Plotting
h_range = np.linspace(0, 0.11, 100)
plt.errorbar(h, z, yerr=dz, fmt='o', label='Data (z vs h)')
plt.plot(h_range, physical_model(h_range, *popt), label='Linear Fit')
plt.xlabel('Height h (m)')
plt.ylabel('z = 2x/t^2 (m/s^2)')
plt.legend()
plt.show()
```

```
Gravity g: 9.77 ± 0.08 m/s^2
Friction mu: 0.01508 ± 0.00005
```


Linear Fit
Data (z vs h)
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
z = 2x/t^2 (m/s^2)
0.00
0.02
0.04
0.06
0.08
0.10
Height h (m)

3

3(a): Falling Object with Quadratic Drag

i. Acceleration and Terminal Velocity

Newton's Second Law with "down" as positive:

$$ma = mg - \frac{1}{2}\rho ACv^2$$

The acceleration a as a function of velocity v:

$$a = \frac{dv}{dt} = g - \frac{\rho AC}{2m}v^2$$

Terminal velocity v_{ter} occurs when $a = 0$:

$$0 = g - \frac{\rho AC}{2m}v_{ter}^2 \implies v_{ter} = \sqrt{\frac{2mg}{\rho AC}}$$

Substituting v_{ter} back into the acceleration equation:

$$\frac{dv}{dt} = g\left(1 - \frac{v^2}{v_{ter}^2}\right)$$

ii. Velocity as a Function of Time

Separating variables and integrating:

$$\int \frac{dv}{1 - \frac{v^2}{v_{ter}^2}} = \int g\,dt$$

Using the identity $\frac{d}{du}\tanh^{-1} u = \frac{1}{1-u^2}$ where $u = \frac{v}{v_{ter}}$:

$$v_{ter}\tanh^{-1}\left(\frac{v}{v_{ter}}\right) = gt + C$$

With initial condition $v(0) = 0$, we find $C = 0$. Solving for $v(t)$:

$$v(t) = v_{ter}\tanh\left(\frac{gt}{v_{ter}}\right)$$

iii. Position as a Function of Time

Integrating velocity to find position $y(t)$:

$$y(t) = \int v(t)\,dt = \int v_{ter}\tanh\left(\frac{gt}{v_{ter}}\right)dt$$

Using the integral of the hyperbolic tangent function:

$$y(t) = \frac{v_{ter}^2}{g} \ln\left|\cosh\left(\frac{gt}{v_{ter}}\right)\right| + C$$

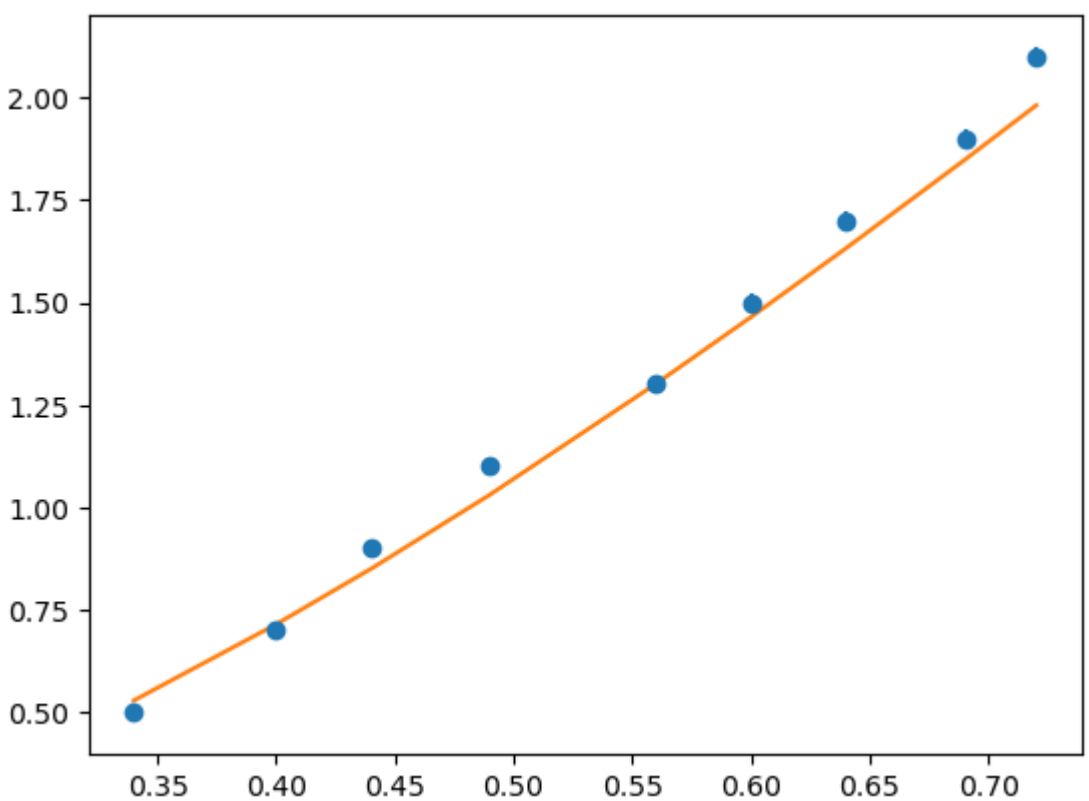
With initial condition $y(0) = 0$, $C = 0$ since $\ln(\cosh(0)) = 0$:

$$y(t) = \frac{v_{ter}^2}{g} \ln\left[\cosh\left(\frac{gt}{v_{ter}}\right)\right]$$

3.b

In [225…

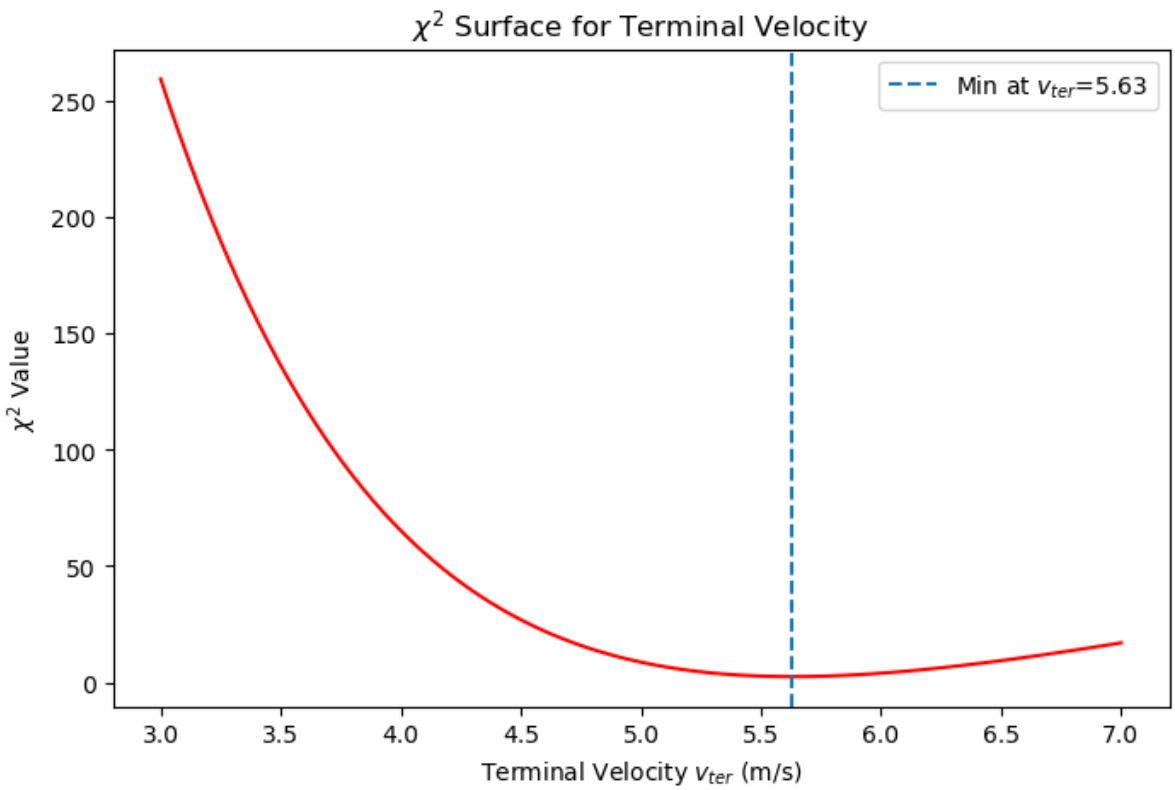
```
from numpy import log, cosh

y=[0.5,0.7,0.9,1.1,1.3,1.5,1.7,1.9,2.1]
t=[0.34, 0.40, 0.44, 0.49, 0.56, 0.60, 0.64, 0.69, 0.72]
dt=0.02

guess=5
g=9.81
def f(t,v_t):
    return v_t**2/g*log(cosh(g*t/v_t))

plt.errorbar(t, y, yerr=dt, fmt='o', label='Measured')
plt.plot(t, f(np.array(t), guess), label='Model')
plt.show()
```



In [224…

```
from sympy import Matrix, symbols, Eq
import numpy as np

y_input=np.array([0.5,0.7,0.9,1.1,1.3,1.5,1.7,1.9,2.1])
t=array([0.34, 0.40, 0.44, 0.49, 0.56, 0.60, 0.64, 0.69, 0.72])
dt=0.02

def y(t,v_t):
    return v_t**2/g*log(cosh(g*t/v_t))

v_t_guesses=np.linspace(3.0,7.0, 100)
chi_sq_vals=[]

for i in v_t_guesses:
    y_predicted = y(t, i)
    # Chi-squared formula: sum of ((observed - expected)^2 / uncertainty^2)
    chi_sq = np.sum(((y_input - y_predicted)**2) / (dy**2))
    chi_sq_vals.append(chi_sq)

min_index = np.argmin(chi_sq_vals)
best_guess=v_t_guesses[min_index]
min_chi_sq = chi_sq_vals[min_index]

print(f"Best V_terminal guess: {best_guess:.2f} m/s")
print(f"Min Chi-Squared: {min_chi_sq:.2f}")

# Plotting the Chi-Squared Parabola
plt.figure(figsize=(8, 5))
plt.plot(v_t_guesses, chi_sq_vals, color='red')
plt.axvline(best_guess, linestyle='--', label=f'Min at $v_{{ter}}$={best_guess:.2
plt.xlabel('Terminal Velocity $v_{ter}$ (m/s)')
plt.ylabel('$\chi^2$ Value')
plt.title('$\chi^2$ Surface for Terminal Velocity')
plt.legend()
plt.show()
```

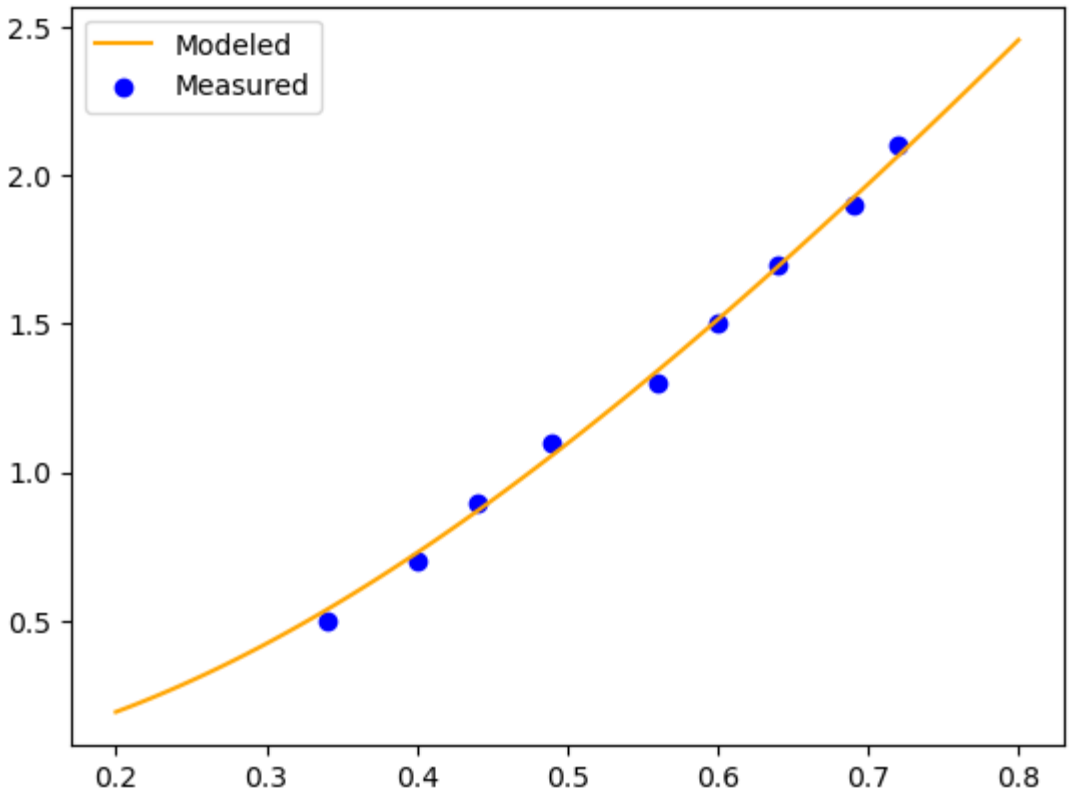
```
Best V_terminal guess: 5.63 m/s
Min Chi-Squared: 2.42
```



3.c

In [235…

```
from numpy import log, cosh, linspace
from matplotlib import pyplot as plt

def f(t,v_t):
    return v_t**2/g*log(cosh(g*t/v_t))

y=[0.5,0.7,0.9,1.1,1.3,1.5,1.7,1.9,2.1]
t=[0.34, 0.40, 0.44, 0.49, 0.56, 0.60, 0.64, 0.69, 0.72]
dt=0.02

t_range= linspace(0.2,0.8,100)

plt.plot(t_range,f(t_range,5.63), label = "Modeled", color = "orange")
plt.scatter(t,y, label ="Measured", color = "blue" )
plt.legend()
plt.show()
```



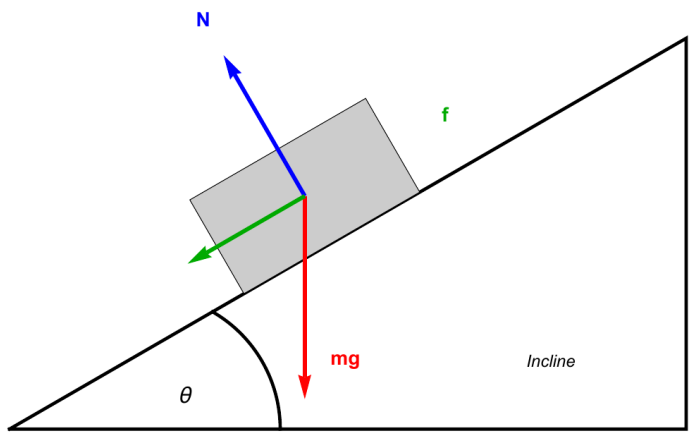
4 & 5

In [236…

```
from pdf2image import convert_from_path
from IPython.display import display

images = convert_from_path(r"C:\Users\miles\BYU-Idaho\11th Semester\336 Advanced
for img in images:
    display(img)
```

In[7]:=
```
Graphics[{(*Define the incline angle*)θ = 30 Degree;
  (*Draw the Inclined Plane*)
  Thickness[0.004], Line[{{0, 0}, {5, 0}, {5, 5 Tan[θ]}, {0, 0}}],
  (*Draw the Cart (rotated to match incline)*)
  Rotate[{EdgeForm[Black], GrayLevel[0.8], Rectangle[{2, 0}, {3.5, 0.8}], Black},
   θ, {0, 0}], (*Force Vectors (originating from center of mass)*)
  {Thickness[0.005], Blue, Arrowheads[0.03], (*Center of mass position*)
   With[{com = RotationTransform[θ, {0, 0}][{2.75, 0.4}]},
    {(*Gravity:mg*)Red, Arrow[{com, com + {0, -1.5}}],
     Text[Style["mg", 14, Bold], com + {0.3, -1.2}], (*Normal Force:N*)Blue,
     Arrow[{com, com + RotationTransform[θ + 90 Degree, {0, 0}][{1.2, 0}]}],
     Text[Style["N", 14, Bold], com + RotationTransform[θ + 90 Degree, {0, 0}][{1.5, 0}]],
     (*Friction Force:f*)Darker[Green],
     Arrow[{com, com + RotationTransform[θ + 180 Degree, {0, 0}][{1.0, 0}]}], Text[
      Style["f", 14, Bold], com + RotationTransform[θ + 180 Degree, {0, 0}][{-1.2, 0}]]]}],
  (*Angle Theta*)Circle[{1, 0}, 1, {2 * θ, 0}], Text[Style["θ", 16], {1.3, 0.25}],
  (*Labels*)Text[Style["Incline", 12, Italic], {4, 0.5}]},
 PlotRange → {{-0.5, 5.5}, {-0.5, 4}},
 ImageSize → Large]
```

Out[7]=



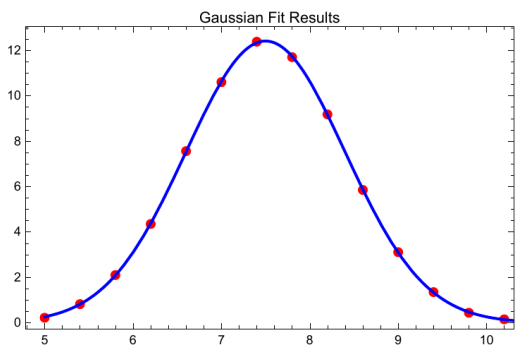
Problem 4.

In[133]:=

```
(*1. Clear variables to avoid conflicts*)
Clear[x, a, mu, sigma, xData, yData, data, nlp, model];

(*2. Define the data*)
xData = {5.0, 5.4, 5.8, 6.2, 6.6, 7.0, 7.4, 7.8, 8.2, 8.6, 9.0, 9.4, 9.8, 10.2};
yData =
  {0.22, 0.82, 2.10, 4.35, 7.56, 10.60, 12.38, 11.70, 9.18, 5.85, 3.11, 1.35, 0.44, 0.15};
data = Transpose[{xData, yData}];

(*3. Define the model and perform the fit*)
model = a * Exp[- (x - mu) ^2 / (2 * sigma^2)];

(*We provide explicit starting values to help the solver*)
nlp = NonlinearModelFit[data, {model, sigma > 0}, {{a, 12}, {mu, 7.5}, {sigma, 1}}, x];

(*4. Output the results*)
Print["Best Fit Parameters: ", nlp["BestFitParameters"]];

(*5. Visualize*)
Show[
 ListPlot[data, PlotStyle → {Red, PointSize[Large]}, PlotLabel → "Gaussian Fit Results"],
 Plot[nlp[x], {x, 5, 10.5}, PlotStyle → Blue], Frame → True, Axes → False]
```

```
Best Fit Parameters: {a → 12.4092, mu → 7.49934, sigma → 0.899733}
```

Out[140]=



Problem 5.

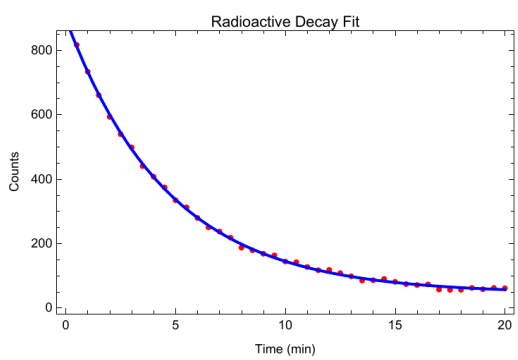
In[160]:=

```
Clear[t, counts, data, model, nlp, N0, lambda, bkg];

(*Time intervals from 0.5 to 20 in steps of 0.5*)
tData = Range[0.5, 20.0, 0.5];

(*Count data from the image*)
counts = {816, 733, 660, 593, 539, 498, 440, 407, 374,
   334, 312, 279, 250, 237, 218, 187, 179, 168, 163, 144, 142, 127, 117,
   118, 108, 98, 85, 86, 90, 81, 74, 71, 73, 57, 56, 56, 62, 58, 62, 61};

data = Transpose[{tData, counts}];
model = n0 * Exp[-lambda * t] + bkg;

nlp = NonlinearModelFit[data, model, {{n0, 900}, {lambda, 0.1}, {bkg, 60}}, t];

(*Extract the results*)
nlp["BestFitParameters"]
Show[ListPlot[data, PlotStyle → Red, PlotLabel → "Radioactive Decay Fit"],
 Plot[nlp[t], {t, 0, 20}, PlotStyle → Blue],
 Frame → True, FrameLabel → {"Time (min)", "Counts"}]
Log[2] / (lambda /. nlp["BestFitParameters"])
```

Out[166]=

```
{n0 → 851.96, lambda → 0.214838, bkg → 45.0613}
```

Out[167]=



Out[168]=

```
3.22637
```


In [ ]: